Vision-Based Line Following Using Convolutional Regression

F211705

Abstract

This report presents a vision-based line-following system implemented on a differential-drive mobile robot. A MobileNet-V3 Small convolutional neural network, pre-trained on ImageNet, is fine-tuned as a regression model to predict the horizontal centre coordinate of a floor track from masked RGB frames captured by a NVIDIA Jetson Orin Nano ZED2i camera. Two training pipelines are evaluated: a Baseline approach using standard batch loading, and an Enhanced approach incorporating weighted random sampling and horizontal flips to mitigate straight-line label imbalance. The Enhanced pipeline reduces validation mean absolute error (MAE) from 3.06 to 2.16 px. Deployed on the physical robot under a proportional controller, the Enhanced model achieves a 100% track completion rate across both directions with 0 manual interventions, compared to 4 off-track events and 1 intervention for the Baseline. Proportional gain (K_P) and lookahead sweeps locate a highly stable steering configuration ($K_P = 0.4$, $K_{\text{angle}} = 15\,\text{px}$) resulting in path following with average lap times of 37.52 s (CW) and 37.85 s (ACW), while speed sweeps identify 0.5–0.7 throttle as the physical stability limit. The complete source code for this project is available at `hhttps://github.com/GeoffreySie/line-following-robot`.

1. Introduction

Autonomous line following is a core robotics task. Classical pipelines rely on hand-crafted thresholding that is brittle to illuminance variation and image noise. Transfer learning from ImageNet-pretrained CNNs provides a more robust alternative, as the pre-trained feature extractor adapts to the target domain with limited labelled data [5].

This work implements a regression-based line follower on the driving robots provided in the lab sessions (Jetson Orin Nano, ZED 2 camera, and Pimoroni Yukon motor controller). A **MobileNet-V3 Small** backbone [1] is fine-tuned to regress the track-centre coordinate c_x from masked RGB frames. Standard uniform training (Baseline) is compared to an Enhanced pipeline adding weighted sampling and horizontal flips. Both drive a proportional controller dispatching UART motor commands in real time.

To evaluate this system, the project is structured around the following four goals and objectives:

1. **Model Fine-Tuning:** Adapt a lightweight deep learning backbone to regress the track-centre coordinate from a custom collected and labelled dataset.
2. **Data Imbalance:** Implement weighted random sampling and horizontal flips to overcome straight-line bias in the training distribution.
3. **Closed-Loop Control:** Design and deploy a real-time proportional steering controller to convert horizontal coordinate predictions into motor commands.
4. **Robustness Sweep:** Experimentally benchmark the steering controller gains (K_P and K_{angle}) and speed limits in both clockwise and anticlockwise configurations to find physical operational sweet spots.

2. Literature Review

2.1 Supervised Learning and Computational Constraints

Recent literature highlights that while traditional supervised learning remains effective for core robotics tasks like obstacle detection and path planning, complex deep learning models on mobile robots are constrained by limited edge computational resources, leading to delayed decisions [8].

To address the computational limitations, researchers have focused on developing lightweight architectures optimised for edge devices. Ye et al. proposed Mobip, a multi-task network utilising a MobileNetV2 backbone designed to simultaneously execute object detection, drivable area segmentation and line detection [9]. The authors found that the MobileNetV2 architecture achieved an inference speed of 58 frames per second (FPS) on a V100 GPU. The authors concluded that the lightweight architecture successfully maintains competitive accuracy across multiple visual perception tasks without needing the computational overhead of typical deep learning models.

2.2 Traditional PID v/s Deep Learning for Line Following

Leal et al. conducted a comparative analysis of a classical Proportional-Integral-Derivative (PID) controller against Long Short-Term Memory (LSTM) and Convolutional Neural Network (CNN) models [4]. The authors found that for basic tracks, the PID controller remained the most efficient and practical, yielding the lowest Mean Absolute Error (MAE) and the highest processing sampling rate at 80 Hz.

On the other hand, the authors also noted that CNNs (which utilised a LeNet-5 architecture for image classification) was computationally heavy and sensitive to environmental noise and lighting variations. However, the CNN demonstrated superior adaptability for complex path navigation, especially in sharp angles

where traditional PID controllers struggled.

2.3 Hybrid Control Strategies

To overcome the limitations of both classical PID and deep learning systems, recent studies have explored hybrid methodologies. Kim et al. developed a hybrid control architecture that integrates a CNN-based behaviour cloning algorithm with a classical PID controller [3]. The authors found that the standalone PID control generated significant oscillation, whereas the standalone behaviour cloning model occasionally failed to adapt to unseen environments.

To resolve these issues, the authors combined the PID method, which directly calculates and minimizes the heading angular error based on the lane's center, with the trajectory generation of the CNN. Their findings revealed that the hybrid architecture reduced the total sum of the heading angular error by up to 53.5% compared to the standalone behaviour cloning model, and improved driving completion times. The authors concluded that fusing deep learning perception with classical PID steering combines the visual adaptability of deep learning with the tracking stability of traditional control systems.

2.4 Identified Gaps and Project Contribution

Despite recent advances in mobile robot control, the existing literature exhibits several gaps. First, studies often evaluate perception models in simulation or on high-end desktop hardware (such as Ye et al.'s use of a V100 GPU [9]), failing to address the severe latency, cooling, and memory constraints of low-power edge computers like the Jetson Orin Nano. Second, comparative analyses frequently assume sensor configurations, neglecting real-world physical constraints such as ZED2i camera mounting offsets that introduce directional biases in clockwise versus anticlockwise driving. Finally, literature rarely presents exhaustive empirical sensitivity sweeps on steering controller lookahead (K_{angle}) and speed settings under straight-line training data biases.

This work addresses these gaps by evaluating a lightweight (∼1.5M parameter) MobileNet-V3 Small regression network on a physical mobile platform, evaluating directional driving asymmetries in both lap directions, and providing a systematic sweep of controller parameters.

3. Methodology

Following literature recommendations, a hybrid control architecture was designed to leverage the complementary strengths of deep learning-based vision and classical feedback control. While pure end-to-end learning exhibits vulnerability to localized steering drift, classical controllers lack the visual robustness to handle severe lighting variations and shadows without brittle hand-crafted filtering [3]. Fusing deep visual perception with a feedback controller reduces heading error oscillations and improves track completion rates compared to standalone systems [3]. This approach also adopts the recommendation of Leal et al. [4], utilizing a high-frequency proportional controller for actuation speed and straight-line stability, while overcoming cornering limitations via deep feature extraction. The hybrid architecture, mapping raw frames to motor execution, consists of: (i) a MobileNet-V3 Small regression frontend predicting the normalized horizontal track-centre coordinate $\hat{c}_x$, and (ii) a proportional steering controller translating this lateral error into discrete UART motor commands (Figure 3.1).

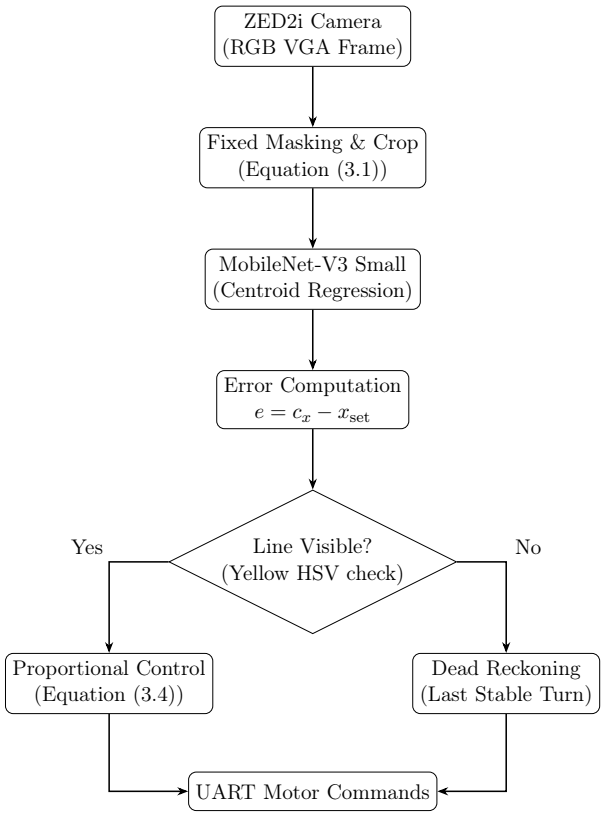

Figure 3.1: Overall Hybrid System Architecture.

3.1 Data Collection & Preprocessing

Data was recorded from seven runs in each track direction using StereoLabs SVO2 format (lossless compression due to the lack of hardware H.264/H.265 encoding on the NVIDIA Jetson Orin Nano), and converted to AVI. Manually pushing the robot along the path eliminated extreme steering oscillations, ensuring the training set represented optimal driving trajectories.

From these runs, 15,145 raw VGA images (672×376 px) were extracted. A fixed mask was applied to each image to suppress background clutter and

constrain the receptive field to the active floor track:

$$M[y,x] = \begin{cases} 0 & y < 188,\ x < 248,\ \text{or } x \geq 604 \\ \text{orig.} & \text{otherwise} \end{cases} \tag{3.1}$$

where $M[y,x]$ is the masked pixel intensity at (y,x), and orig. is the original pixel intensity. The mask is applied identically during training and inference. The receptive field is shifted rightwards because the physical ZED2i camera has a slight leftward mounting skew, making the line appear right-biased even when the robot is centered on the track.

3.2 Dataset Construction & Splits

After masking, frames are converted to greyscale and thresholded at intensity 200. The regression label $\hat{c}_x \in [0,1]$ represents the normalized horizontal centre of mass of active white pixels $\mathcal{W}$ representing the track lane:

$$\hat{c}_x = \frac{c_x}{W}, \quad c_x = \frac{1}{|\mathcal{W}|} \sum_{(y,x)\in\mathcal{W}} x, \quad W = 672 \tag{3.2}$$

where c_x is the absolute horizontal pixel centroid, $W = 672\,\text{px}$ is the image width, and $|\mathcal{W}|$ is the count of active thresholded pixels. Frames with $|\mathcal{W}| = 0$ are discarded, yielding 15,145 frames. The dataset was split 80/20 into training (12,116 frames) and validation (3,029 frames) sets using random seed 42. Training frames were augmented online using ColorJitter (brightness, contrast, and saturation factors of 0.2) to simulate lighting variations. The Enhanced pipeline additionally used random horizontal flips to double curved track examples and balance the training distribution. Since straight-line driving labels are naturally over-represented, we introduce the weighted sampling strategy detailed in Section 3.4.

3.3 Model Selection and Training

We selected MobileNet-V3 Small [1], pre-trained on ImageNet [7], as the backbone. This selection directly addresses the computational constraints of deploying deep learning on resource-constrained mobile platforms highlighted by Rybczak et al. [8]. With only ∼1.5M parameters, this lightweight CNN ensures low latency and high-frequency real-time edge execution, aligning with the design principles of Ye and Zhang [9] who demonstrated that MobileNet backbones achieve high inference frame rates on edge devices.

To adapt the backbone for lateral centroid regression, the 1000-class classification head is replaced by a single linear regression neuron ($\mathbf{z} \in \mathbb{R}^{576}$). All parameters were fine-tuned end-to-end using the AdamW optimiser [6] and SmoothL1 loss [2]. AdamW decoupled weight decay provides superior regularization, preventing overfitting during domain transfer. SmoothL1 loss is selected because mathematically generated ground-truth centroids can exhibit high-frequency noise from masking boundaries. By acting as an L1 loss for large errors and an L2 loss for small errors, SmoothL1 is robust to outliers, preventing gradient explosion and stabilizing regression optimization [2].

3.4 Training Pipelines

Due to the specific nature of the data collected, we employed several improvements to our model training pipeline. To document the effect of these improvements, we designed two pipelines: A Base pipeline and an Enhanced pipeline. We compare the results in Chapter 4. Both pipelines resize inputs to 224×224, apply ColorJitter augmentation, use batch size 128, and early stopping. Table 3.1 shows their key differences.

For the Base pipeline, frames were loaded into a standard `TensorDataset` (100 epochs, no LR schedule). The Enhanced pipeline adds weighted sampling, dividing labels into 10 buckets. Each sample has weight $w_i = 1/\text{count}(b_i)$ to balance straight and curved frames. It also uses a `ReduceLROnPlateau` scheduler and trains for 500 epochs.

Table 3.1: Pipeline comparison.

Setting	Base	Enhanced
Weighted sampling	No	Yes
Max epochs	100	500
Patience	10	15
LR scheduler	–	ReduceLROnPlateau

Early stopping monitored validation MAE after each epoch. If validation MAE improved by more than 10^{-6}, the no-improvement counter was reset and model weights were saved as the best checkpoint. Otherwise, the counter was incremented. Training terminated when this counter reached the pipeline's patience value, preventing overfitting and ensuring final evaluation used the optimal validation checkpoint.

3.5 Inference Pipeline

3.5.1 Proportional Control

Frames are masked, converted to RGB, and processed on the GPU. The model predicts the normalized coordinate $\hat{c}_{\text{pred}}$, scaled to pixel space as $c_x = \hat{c}_{\text{pred}} \times W$. The predicted lane centre c_x is compared with the reference setpoint $x_{\text{set}} = 420\,\text{px}$ (offset to compensate for physical mounting skew) to compute the lateral tracking error e:

$$e = c_x - x_{\text{set}} \tag{3.3}$$

where e is the horizontal tracking error in pixels.

A proportional controller calculates the steering com-

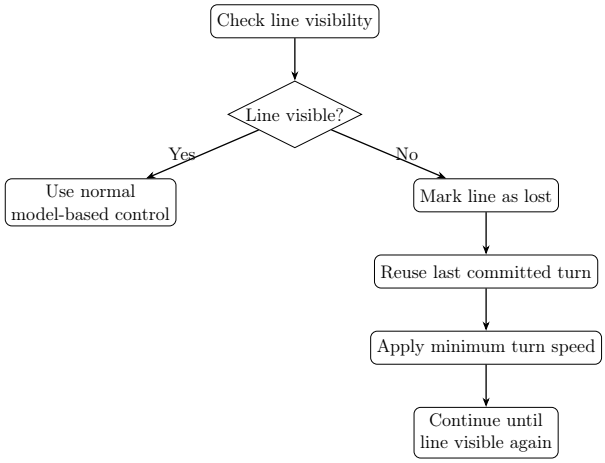

Figure 3.2: Dead Reckoning for Line Loss.

mand velocity v:

$$v = \text{clip}\left(\frac{K_P|e|}{W/2}, s_{\min}, s_{\max}\right) \tag{3.4}$$

where v is the proportional velocity command, K_P is the proportional gain, $W = 672$, and $[s_{\min}, s_{\max}]$ are speed limits. If $|e| \leq 40\,\text{px}$, the robot drives straight `forward`; otherwise, it steers `left` or `right` based on the sign of e.

3.5.2 Dead Reckoning

A dead reckoning failsafe handles brief track visibility losses, particularly in sharp corners (Figure 3.2).

If the visible yellow pixel count falls below a set threshold, the controller marks the line as lost and reuses the last committed steering direction. To prevent high-frequency noise from destabilizing steering, a new turning direction must persist for at least 0.2 s to replace the previous command.

4. Results

4.1 Model Training

Both models converged under early stopping. Figure 4.1 shows the training and validation SmoothL1 loss curves for the Base and Enhanced pipelines. Both curves exhibit rapid loss reduction within the first 5 epochs, showing that the pre-trained MobileNet-V3 Small backbone adapts quickly. Base early-stopped at 34 epochs, whereas Enhanced trained for 127 epochs. Base's validation loss fluctuated on the plateau, but Enhanced remained stable, showing the benefit of the ReduceLROnPlateau scheduler. Enhanced achieved a final validation MAE of 2.16 px, a 29.4% reduction over Base's 3.06 px.

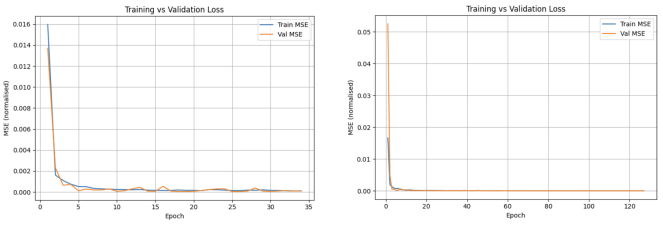

Figure 4.1: SmoothL1 loss curves. Left: Base, Right: Enhanced.

Figure 4.2 compares predicted and ground-truth validation set distributions. Both pipelines show that the models mostly predict labels correctly. However, the Enhanced pipeline clearly shows an improvement over the Base pipeline, where predictions more accurately follow the ground-truth distribution.

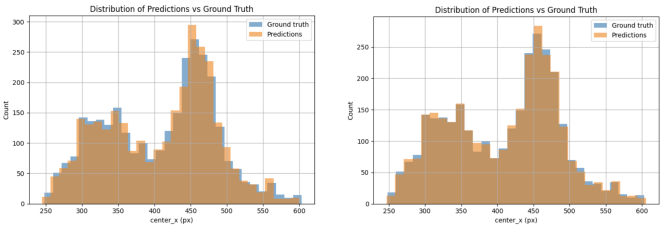

Figure 4.2: Predicted vs. ground-truth c_x distributions. Left: Base, Right: Enhanced.

4.2 Real-World Driving Evaluation

In order to test our models, we deployed them onto the mentioned robots and evaluation their lap completion times, off-track events, interventions needed, and sensitivity to controller settings. Results are reported separately for clockwise and anticlockwise laps so that directional weaknesses are not hidden.

4.2.1 Live Driving Performance

Table 4.1: Live driving performance comparison under default configuration.

Dir.	Lap	Time (s)	Off-Track	Intv.
Baseline Model				
CW	1	38.53	1	0
CW	2	38.22	0	0
ACW	1	41.97	2	1
ACW	2	38.97	1	0
Total	—	—	4	1
Enhanced Model				
CW	1	38.04	1	0
CW	2	38.11	1	0
ACW	1	37.97	1	0
ACW	2	39.02	1	0
Total	—	—	4	0

Comparing the two models in Table 4.1 reveals a significant improvement. The Baseline model cut corners

too closely, leaving the track four times and requiring a manual intervention during the first ACW lap. This straight-line bias arose because the standard dataset was dominated by straight driving, causing the model to predict central coordinates ($c_x \approx 420\,\text{px}$) and react sluggishly in bends. The Enhanced model resolved this using weighted random sampling and horizontal flips to balance curved track frames. Consequently, the robot steered responsively, completing all laps successfully without interventions and maintaining highly stable lap times of 38 s.

Additionally, both models took slightly longer on anticlockwise laps. This difference is likely because the ZED2i camera is physically mounted slightly to the left of the robot's center line. This offset makes left turns look tighter in the camera's view, making them slightly harder to track smoothly.

4.2.2 Controller Sensitivity

Table 4.2: Controller sensitivity sweep with the Enhanced model fixed.

K_P	K_{angle} (px)	Dir.	Time (s)	Off	Intv.
0.3	30	CW	55.04	5	3
0.3	30	ACW	60.55	6	5
0.4	30	CW	37.82	1	0
0.4	30	ACW	38.12	0	0
0.5	30	CW	50.45	3	3
0.5	30	ACW	51.90	4	3
0.4	15	CW	37.52	0	0
0.4	15	ACW	37.85	0	0
0.4	45	CW	49.65	4	3
0.4	45	ACW	48.88	5	3

The sensitivity sweep in Table 4.2 distinguishes model error from controller-induced instability. Under sluggish proportional gain ($K_P = 0.3$), the robot steered too weakly to negotiate bends, driving straight off the track and requiring up to 5 interventions. Conversely, high gain ($K_P = 0.5$) induced aggressive overcorrections and side-to-side hunting oscillations on straightaways, leading to control loss and off-track failures in curves. $K_P = 0.4$ achieved the ideal balance. For lookahead, a smaller value ($K_{\text{angle}} = 15\,\text{px}$) yielded the fastest, most stable driving (0 off-track events). Increasing it to $K_{\text{angle}} = 45\,\text{px}$ caused the robot to turn too early, resulting in severe corner cutting and off-track failures.

4.2.3 Speed Robustness

Table 4.3: Speed robustness study under the default controller configuration.

Min	Max	Dir.	Lap Time (s)	Off	Intv.
0.3	0.5	CW	57.46	0	0
0.3	0.5	ACW	59.86	1	0
0.5	0.7	CW	38.04	1	0
0.5	0.7	ACW	38.55	1	0
0.7	0.9	CW	38.31	5	2
0.7	0.9	ACW	40.50	4	3

The speed sweep in Table 4.3 highlights physical momentum constraints. At low speeds (0.3–0.5), the robot was highly stable due to low momentum and ample processing time per frame, but suffered slow lap times (60 s). High speeds (0.7–0.9) caused severe instability and multiple off-track events. The increased momentum, combined with processing and communication latency, caused the robot to slide out of curves before commands could be executed, while also amplifying steering adjustments into unstable wobbles. The default range (0.5–0.7) proved optimal, balancing safety and speed (38 s).

5. Discussion

5.1 Analysis of Experimental Results

Evaluating the real-world performance shows a direct connection between offline validation error and actual driving stability. The Enhanced model's lower validation error of 2.16 px (compared to 3.06 px for the Baseline) successfully eliminated all manual interventions in live runs. The Baseline model suffered from straight-line bias because the original training dataset was dominated by straight track frames. Weighted sampling and horizontal-flip augmentations in the Enchaned pipeline during training resolved this data bias, forcing the network to learn active turning features instead of predicting near-center coordinates.

However, a highly accurate perception model can still fail if the controller parameters are poorly matched to the physical system. A low proportional gain ($K_P = 0.3$) causes sluggish steering, resulting in severe understeer and track departures. Conversely, a high gain ($K_P = 0.5$) leads to aggressive overcorrections and side-to-side hunting oscillations. Similarly, a large lookahead angle ($K_{\text{angle}} = 45\,\text{px}$) makes the controller turn too early. The optimal parameters ($K_P = 0.4, K_{\text{angle}} = 15\,\text{px}$) successfully balance steering speed and path tracking. Finally, speed sweeps indicate that high momentum (0.7–0.9), combined with frame-processing latency and UART transmission delays, causes the robot to skid off track before turns can be physically executed, validating the de-

fault speed range of 0.5–0.7 as the physical sweet spot.

5.2 System Limitations and Future Work

Several system limitations must be addressed to transition from a prototype to a fully autonomous platform. The preprocessing pipeline relies on manual, fixed boundary masking. If ambient lighting changes or the camera shifts, the mask fails, which could be resolved using adaptive edge thresholding. Additionally, the system lacks temporal context because it processes frames independently, ignoring past motion and causing minor steering jitter. Implementing a moving average on steering errors or a recurrent neural network (LSTM) head would smooth performance. Finally, the coarse steering protocol relies on discrete left-straight-right commands, preventing smooth differential speed adjustments and causing chattering on gradual bends. A continuous PID steering controller should be developed to overcome this issue.

6. Conclusion

In conclusion, a complete vision-based line-following pipeline was successfully deployed. The four objectives were met as follows:

1. **Model Fine-Tuning (Met):** The MobileNet-V3 Small backbone adapted rapidly to the regression task, converging efficiently and providing high horizontal coordinate accuracy.
2. **Data Imbalance (Met):** The Enhanced training pipeline successfully resolved dataset bias. Balanced sampling and horizontal flips reduced validation MAE from 3.06 px to 2.16 px and completely eliminated manual driving interventions.
3. **Closed-Loop Control (Met):** The proportional controller translated visual predictions into real-time steering. An optimal setting of $K_P = 0.4$ and $K_{\text{angle}} = 15\,\text{px}$ achieved smooth, stable track tracking.
4. **Robustness Sweep (Met):** Sweeps mapped out clear physical operating limits, confirming that a default speed window of 0.5–0.7 for was optimal for driving safety.

For future work, calculated speed commands should be implemented to automatically slow the robot down in sharp corners, and temporal context (such as moving averages or LSTM layers) should be introduced to smooth steering jitter.

Bibliography

[1] A. Howard, M. Sandler, G. Chu, L.-C. Chen, B. Chen, M. Tan, W. Wang, Y. Zhu, R. Pang, V. Vasudevan, Q. V. Le, and H. Adam. Searching for MobileNetV3. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 1314–1324, 2019.

[2] P. J. Huber. Robust estimation of a location parameter. *The Annals of Mathematical Statistics*, 35(1):73–101, 1964.

[3] H.-G. Kim, J. Myoung, S. Lee, K.-M. Park, and D. Shin. Design of ai-powered hybrid control algorithm of robot vehicle for enhanced driving performance. *IEEE Embedded Systems Letters*, 16(2):94–97, 2024. doi: 10.1109/LES.2023.3257774.

[4] H. M. Leal, R. S. Barbosa, and I. S. Jesus. Control of a mobile line-following robot using neural networks. *Algorithms*, 18(1), 2025. ISSN 1999-4893. doi: 10.3390/a18010051.

[5] Y. LeCun, Y. Bengio, and G. Hinton. Deep learning. *Nature*, 521(7553):436–444, 2015.

[6] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019. URL `https://openreview.net/forum?id=Bkg6RiCqY7`.

[7] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, A. C. Berg, and L. Fei-Fei. ImageNet large scale visual recognition challenge. *International Journal of Computer Vision*, 115(3):211–252, 2015.

[8] M. Rybczak, N. Popowniak, and A. Lazarowska. A survey of machine learning approaches for mobile robot control. *Robotics*, 13(1), 2024. ISSN 2218-6581. doi: 10.3390/robotics13010012.

[9] M. Ye and J. Zhang. Mobip: a lightweight model for driving perception using mobilenet. *Frontiers in Neurorobotics*, Volume 17 - 2023, 2023. ISSN 1662-5218. doi: 10.3389/fnbot.2023.1291875.